

# YOLO-OBB Model Development for Slide Analysis

## Project Documentation

December 6, 2025

## 1 Component Detection YOLO Model

I developed a custom YOLO-OBB (Oriented Bounding Box) model to help detect components on lecture slides. Since diagrams often have rotated shapes/images so standard bounding boxes aren't accurate enough. OBB allows the model to be more accurate on these types of objects.

### 1.1 Target Class

Right now, the model is focused on a single class to get every visual component:

- **diagram\_node:** This includes all the visual parts of a slide, like the nodes of a diagram, shapes, images, etc...

### 1.2 Dataset Construction

Finding good annotated datasets for presentation slides is difficult, so I built a hybrid dataset to cover different visual styles.

#### 1.2.1 Data Sources

- **AI2D (Scientific Diagrams):** I used the AI2D dataset to get high-quality bboxes for nodes in scientific diagrams.
- **Synthetic Generation:** To make the model work better on technical slides with flowcharts, I wrote a script to generate synthetic data. It places random elements like cloud icons, flowchart boxes, and stylized shapes onto slide-like backgrounds. This helps the model learn to recognize objects in diagrams and flowcharts more related to the computer science field.

| Split      | Count | Percentage |
|------------|-------|------------|
| Train      | 6320  | 80%        |
| Validation | 810   | 10%        |
| Test       | 814   | 10%        |

Table 1: Dataset Splits for Logo/Image Model

### 1.3 Training and Refinement

1. **Initial Training:** I started by training on the mix of AI2D and my synthetic data.  
mAP50: 0.995  
mAP50-95: 0.98

I then ran this model on actual lecture slides from my classes. The performance was not good.

2. **Fine-Tuning:** I manually corrected the detections on 100 real slides using the model trained above. Retraining on this small "gold standard" set significantly improved the results on actual slides.
- mAP50: 0.95  
mAP50-95: 0.87

## 2 Logo vs. Image YOLO model

Another issue I had to solve was distinguishing between a company logo and a regular content image (like a photo or screenshot). I trained a separate model just for this.

### 2.1 Dataset Overview

I generated this dataset synthetically.

- **Logo (0):** Brand marks and logos.
- **Images (1):** Any other kind of image.

I used logos from the *Logo-2k+* dataset and images from the *SlideVQA* dataset. The generation script places 2 to 6 objects on a gradient background and adds random text titles. This prevents the model from relying on simple shortcuts (like assuming a logo always appears on a white background).

### 2.2 Results

The dataset is fairly balanced (48% logos vs 52% images). The model learned the difference very quickly.

| Split      | Count | Percentage |
|------------|-------|------------|
| Train      | 1,564 | 70%        |
| Validation | 333   | 15%        |
| Test       | 329   | 15%        |

Table 2: Dataset Splits for Logo/Image Model

On the test set, the performance is very good:

- **mAP50:** 0.998
- **mAP50-95:** 0.995

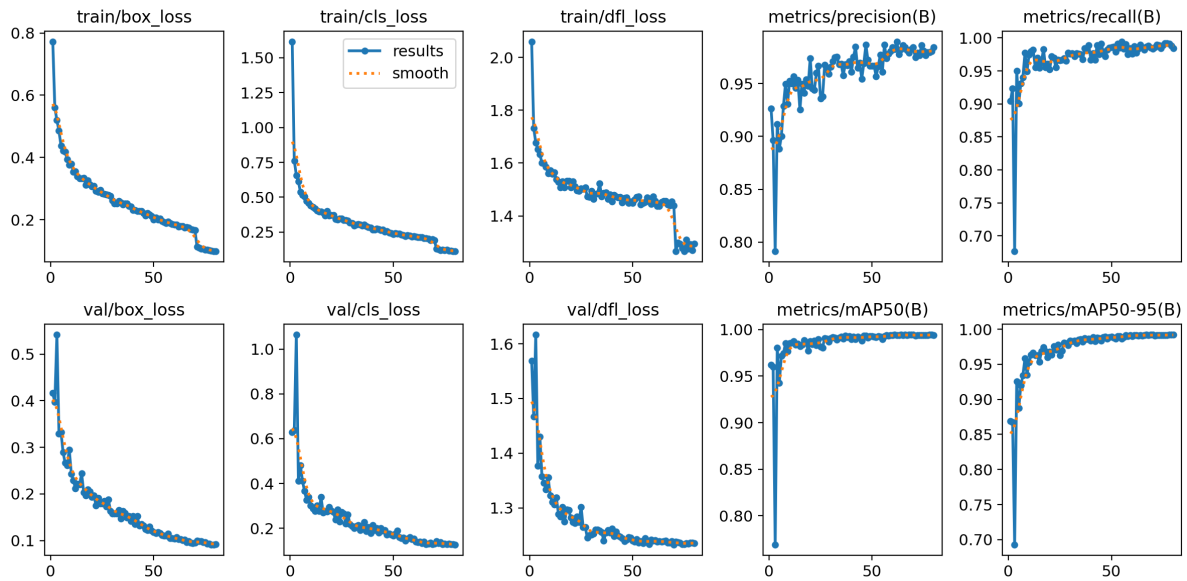


Figure 1: Training metrics for the Logo vs. Image model. As shown in the graphs, the loss drops quickly and the precision/recall scores hit nearly 1.0 very early in the training.

### 3 Bonus Point 4: Data Collection and Enhancement

#### 3.1 Original Data and Annotation

The public datasets we started with were helpful, but they did not fully match the style of real computer science lecture slides. To close this gap, I created a small but high-quality dataset myself.

- **Manual Annotation:** I collected around 100 real lecture slides from classes I previously took and created my own annotated dataset. To speed up the process, I first ran my partially trained model (trained on synthetic data and public datasets) on these slides, and then manually corrected any mistakes it made. This allowed me to capture rotated images, illustrations, and diagram components more accurately .
- **Why This Was Needed:** These hand-annotated slides served as a “gold standard” and were especially important during the fine-tuning stage (Section 1.3). They helped connect the cleaner AI2D diagrams with the much messier layout of actual course material.

#### 3.2 Simulation and Augmentation Strategy

1. **Synthetic Diagram Generation:** I wrote a Python script that generates CS-style diagrams by randomly placing icons (clouds, databases, rhombuses, etc.) and connecting them with arrows.
2. **Slide Composition for Logos vs. Images:** I built a separate generator that places images or logos on different backgrounds, adds text, and varies lighting and colors. This produced a high-variance dataset without needing to label thousands of samples.

#### 3.3 Challenges and Solutions

One major issue throughout the project was domain shift. Models pre-trained on AI2D worked well on simple white diagrams but struggled on real lecture slides.

- **Issue:** The model started relying on background cues instead of the object itself, for example treating “white space” as a signal for logos.
- **Fix:** The synthetic augmentations (different gradient backgrounds, random text) forced the model to ignore the background and actually pay attention to shapes, edges, and textures. After this change, the model performed noticeably better on actual slides.

### 3.4 Dataset Distinctness

Each model was trained on its own independent dataset.

- **Task 1 — Component Detection:** Trained using AI2D plus the synthetic flowchart diagrams. Fine tuned on real slides. This dataset focuses only on structural elements (nodes, images, blocks).
- **Task 4 — Logo Classification:** Trained using Logo-2k+ and SlideVQA, along with my synthetic slide generator.